

Question Number : 94 Question Id : 6406531252049 Question Type : MSQ

Correct Marks : 3 Max. Selectable Options : 0

Question Label : Multiple Select Question

Which of the following statements are correct based on the KKT conditions and the solution to the optimization problem?

Options :

6406534218052. ✓ $x = y = \frac{\alpha}{2}$

6406534218053. ✗ $z = w = \alpha$

6406534218054. ✓ $\frac{3\beta}{8} \geq \frac{1}{4} - \delta$

6406534218055. ✗ $\frac{3\beta}{8} \leq \frac{1}{4} - \delta$

Question Number : 95 Question Id : 6406531252050 Question Type : SA

Correct Marks : 3

Question Label : Short Answer Question

If $\delta > \frac{1}{4}$ then find the value of x .

Response Type : Numeric

Evaluation Required For SA : Yes

Show Word Count : Yes

Answers Type : Equal

Text Areas : PlainText

Possible Answers :

0.25

Java

Section Id :	64065387635
Section Number :	5
Section type :	Online
Mandatory or Optional :	Mandatory
Number of Questions :	24
Number of Questions to be attempted :	24

Section Marks :	100
Display Number Panel :	Yes
Section Negative Marks :	0
Group All Questions :	No
Enable Mark as Answered Mark for Review and Clear Response :	No
Section Maximum Duration :	0
Section Minimum Duration :	0
Section Time In :	Minutes
Maximum Instruction Time :	0
Sub-Section Number :	1
Sub-Section Id :	640653194517
Question Shuffling Allowed :	No

Question Number : 96 Question Id : 6406531252051 Question Type : MCQ

Correct Marks : 0

Question Label : Multiple Choice Question

THIS IS QUESTION PAPER FOR THE SUBJECT "DIPLOMA LEVEL : PROGRAMMING CONCEPTS USING JAVA (COMPUTER BASED EXAM)"

ARE YOU SURE YOU HAVE TO WRITE EXAM FOR THIS SUBJECT?

CROSS CHECK YOUR HALL TICKET TO CONFIRM THE SUBJECTS TO BE WRITTEN.

(IF IT IS NOT THE CORRECT SUBJECT, PLS CHECK THE SECTION AT THE TOP FOR THE SUBJECTS REGISTERED BY YOU)

Options :

6406534218057.  YES

6406534218058.  NO

Sub-Section Number :	2
Sub-Section Id :	640653194518
Question Shuffling Allowed :	Yes

Question Number : 97 Question Id : 6406531252052 Question Type : MCQ

Correct Marks : 4

Question Label : Multiple Choice Question

Consider the code given below.

```
class Shared {
    static int count = 2000;
}
class Subtractor extends Thread {
    public void run() {
        for (int i = 0; i < 1000; i++) {
            Shared.count--;
        }
    }
}
public class Test {
    public static void main(String[] args) throws InterruptedException {
        Thread t1 = new Subtractor();
        Thread t2 = new Subtractor();
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.print(Shared.count);
    }
}
```

Choose the correct option.

Options :

- 6406534218059. ✖ The code always prints 2000.
- 6406534218060. ✖ The code prints any number in the range 1000 to 2000
- 6406534218061. ✔ The code prints any number in the range 0 to 1000
- 6406534218062. ✖ The code always prints 1000.

Question Number : 98 Question Id : 6406531252053 Question Type : MCQ

Correct Marks : 4

Question Label : Multiple Choice Question

Consider the following Java code that uses chained exceptions.

```
class PaymentException extends Exception {
    public PaymentException(String message) {
        super(message);
    }
}

class OrderProcessingException extends Exception {
    public OrderProcessingException(String message) {
        super(message);
    }
}

class OrderService {
    public void placeOrder() throws OrderProcessingException {
        try {
            // if(payment fails)
            throw new PaymentException("Payment not done");
            // else process the order
        } catch (PaymentException e) {
            OrderProcessingException oe = new OrderProcessingException
                ("Order Failed");

            oe.initCause(e);
            throw oe;
        }
    }
}

public class Test {
    public static void main(String[] args) {
        OrderService orderService = new OrderService();
        try {
            orderService.placeOrder();
        } catch (OrderProcessingException e) {
            System.out.println(e.getMessage());
            System.out.println("Root cause: " + e.getCause().getMessage());
        }
    }
}
```

Which of these could be the output of this code?

Options :

Payment not done

6406534218063. ✖ Root cause: Order Failed

Order Failed

6406534218064. ✔ Root cause: Payment not done

6406534218065. ✖

Root cause: Payment not done

6406534218066. ✖ Root cause: Order Failed

Question Number : 99 Question Id : 6406531252054 Question Type : MCQ

Correct Marks : 4

Question Label : Multiple Choice Question

Consider the two Java files given below.

Animal.java:

```
package creatures;
class Animal {
    private String name;
    private String habitat;
    private int legs;
    // Constructor to initialize instance variables
    public String getName(String name) {
        return name;
    }
    private String getHabitat(String habitat) {
        return habitat;
    }
    protected int getLegs(int legs) {
        return legs;
    }
    public int showLegs(int legs) {
        return getLegs(legs);
    }
}
class Bird extends Animal {
    public void info() {
        System.out.println("This is a Bird.");
    }
    public void displayHabitat() {
        System.out.println("Bird's habitat: " + getHabitat("Trees"));
    }
}
```

Test.java:

```
package creatures;
public class Test {
    public static void main(String[] args) {
        Animal myAnimal = new Animal();
        System.out.println("Number of legs: " + myAnimal.showLegs(4));
        System.out.println(myAnimal.getHabitat("Forest")); // LINE 1
        System.out.println(myAnimal.getLegs(4)); // LINE 2
    }
}
```

Choose the correct option.

Options :

6406534218067. ✖ LINE 1 and LINE 2 generate compilation errors.

6406534218068. ✔ Only LINE 1 generates compilation error.

6406534218069. ✖ Only LINE 2 generates compilation error.

6406534218070. ✖ No compilation error

Question Number : 100 Question Id : 6406531252056 Question Type : MCQ

Correct Marks : 4

Question Label : Multiple Choice Question

Consider the Java code given below.

```
class TemperatureConverter {
    public <T> void convert(T value) { // LINE 1
        System.out.println("Converting: " + value);
    }
    public <U extends Number> void convert(U value) { // LINE 2
        System.out.println("C to F: " + (value.doubleValue() * 9/5 + 32));
    }
    public <T extends Number> void convert(T value) { // LINE 3
        System.out.println("C to K: " + (value.doubleValue() + 273.15));
    }
}

public class Test {
    public static void main(String[] args) {
        TemperatureConverter temp = new TemperatureConverter();
        temp.convert(25);
        temp.convert(100.5);
        temp.convert("Cold");
    }
}
```

Choose the correct option.

Options :

6406534218075. ✖ LINE 2 and LINE 3 generate compilation errors because method convert() is already defined.

6406534218076. ✖ LINE 1 and LINE 3 generate compilation errors because method convert() is already defined.

Only LINE 3 generates a compilation error because method convert() is already defined.  6406534218077.

This program generates output:

C to F: 77.0

C to K: 373.65

6406534218078.  Converting: Cold

Question Number : 101 Question Id : 6406531252057 Question Type : MCQ

Correct Marks : 4

Question Label : Multiple Choice Question

Consider the code given below.

```
class Person {
    public void work() {
        System.out.println("works as a person.");
    }
}
class Teacher extends Person {
    public void work() {
        System.out.println("Works as teacher.");
    }
    public void grade() {
        System.out.println("Grading papers.");
    }
}
class MathTeacher extends Teacher {
    public void solveEquation() {
        System.out.println("Solving complex equations.");
    }
}
public class SchoolTest {
    public static void main(String[] args) {
        Teacher t = new MathTeacher(); // LINE 1
        t.work();
        t.grade(); // LINE 2
        t.solveEquation(); // LINE 3
    }
}
```

Choose the correct option.

Options :

This program generates the output:

Works as teacher.

Grading papers.

Solving complex equations.

6406534218079. ✖

LINE 1 generates compilation error because a variable of type Teacher cannot refer to an object of type MathTeacher.

6406534218080. ✖

LINE 2 generates compilation error because the method grade() is not defined in the MathTeacher class.

6406534218081. ✖

LINE 3 generates compilation error because the method solveEquation() is not defined in the Teacher class.

6406534218082. ✔

Question Number : 102 Question Id : 6406531252058 Question Type : MCQ

Correct Marks : 4

Question Label : Multiple Choice Question

Consider the Java code given below.

```
class Phone {
    String model;
    public Phone(String m) {
        model = m;
    }
    public String toString() {
        return "Model: " + model;
    }
}

class Smartphone extends Phone {
    int storage;
    public Smartphone(String m, int s) {
        super(m);
        storage = s;
    }
    public Smartphone(Smartphone p) {
        super(p.model);
        storage = p.storage;
    }
    public String toString() {
        return super.toString() + ", Storage: " + storage + "GB";
    }
}

public class Store {
    public static void main(String args[]) {
        Phone p1 = new Smartphone("Galaxy S23", 128);
        Phone p2 = new Smartphone((Smartphone) p1);
        p2.model = "iPhone 15";
        System.out.println(p1 + "\n" + p2);
    }
}
```

What will the output be?

Options :

Model: iPhone 15

6406534218083. ✖ Model: iPhone 15

Model: iPhone 15, Storage: 128GB

6406534218084. ✖ Model: iPhone 15, Storage: 128GB

Model: Galaxy S23, Storage: 128GB

6406534218085. ✔ Model: iPhone 15, Storage: 128GB

6406534218086. ✖

Model: Galaxy S23

Model: iPhone 15

Question Number : 103 Question Id : 6406531252059 Question Type : MCQ

Correct Marks : 4

Question Label : Multiple Choice Question

Consider the Java code given below.

```
abstract class SmartDevice {
    abstract void operate();
    void connect() { // LINE 1
        System.out.println("Connecting smart device");
    }
}

class SmartLight extends SmartDevice {
    void operate() {
        System.out.println("Turning on smart light");
    }
    void connect() {
        System.out.println("Connecting smart light to Wi-Fi");
    }
}

class SmartThermostat extends SmartDevice {
    void operate() {
        System.out.println("Setting smart thermostat");
    }
    void connect() {
        System.out.println("Connecting thermostat to network");
    }
}

public class SmartHome {
    public static void main(String[] args) {
        SmartDevice d1 = new SmartLight(); // LINE 2
        SmartDevice d2 = new SmartThermostat(); // LINE 3
        d1.connect();
        d1.operate();
        d2.connect();
        d2.operate();
    }
}
```

Choose the correct option.

Options :

6406534218087. ✖

LINE 1 generates a compilation error because an abstract class must contain only abstract methods.

LINE 2 and LINE 3 generate compilation errors because a reference variable of type SmartDevice cannot store the objects of type SmartLight and SmartThermostat.

6406534218088. ✖

This program generates the output:

Connecting smart light to Wi-Fi
Turning on smart light
Connecting thermostat to network
Setting smart thermostat

6406534218089. ✔

This program generates the output:

Connecting smart device
Turning on smart light
Connecting smart device
Setting smart thermostat

6406534218090. ✖

Question Number : 104 Question Id : 6406531252060 Question Type : MCQ

Correct Marks : 4

Question Label : Multiple Choice Question

Consider the Java code given below.

```
interface EventBooking {  
    void bookEvent();          //LINE 1  
    abstract void cancelEvent();  
}  
  
interface PaymentProcessing {  
    abstract void processPayment(double amount);  
}  
  
class EventManager implements EventBooking, PaymentProcessing {    //LINE 2  
    public void bookEvent() {  
        System.out.println("Event booked successfully");  
    }  
  
    public void cancelEvent() {  
        System.out.println("Event cancelled successfully");  
    }  
  
    public void processPayment(double amount) {  
        System.out.println("Payment of " + amount + " processed");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        EventBooking m = new EventManager();  
        m.bookEvent();  
        m.cancelEvent();  
        m.processPayment(150.0);    //LINE 3  
    }  
}
```

What will the output be?

Options :

6406534218091. ✖ LINE 1 generates compilation error as bookEvent() is not declared as abstract

6406534218092. ✖ LINE 2 generates compilation error as a class cannot implement multiple interfaces

This program generates the output:

Event booked successfully
Event cancelled successfully

6406534218093. ✖ Payment of 150.0 processed

6406534218094. ✓ LINE 3 generates compilation error as processPayment() method is not accessible

Question Number : 105 Question Id : 6406531252061 Question Type : MCQ

Correct Marks : 4

Question Label : Multiple Choice Question

Consider the Java code given below.

```
import java.util.*;
public class Test {
    public static void main(String[] args) {
        Map<String, Integer> bonus2023 = new TreeMap<String, Integer>();
        bonus2023.put("Arun", 5000);
        bonus2023.put("Deepa", 4500);
        bonus2023.put("Manoj", 6000);
        bonus2023.put("Lekha", 5500);

        Map<String, Integer> bonus2024 = new TreeMap<String, Integer>();
        bonus2024.put("Arun", 5200);
        bonus2024.put("Deepa", 4800);
        bonus2024.put("Manoj", 6200);
        bonus2024.put("Lekha", 5700);

        Map<String, Integer> totalBonus = new TreeMap<String, Integer>();

        for (Map.Entry<String, Integer> b1 : bonus2023.entrySet())
            totalBonus.put(b1.getKey(), b1.getValue());

        for (Map.Entry<String, Integer> b2 : bonus2024.entrySet())
            totalBonus.merge(b2.getKey(), b2.getValue(), Integer::sum); // LINE 1

        System.out.println(totalBonus);
    }
}
```

Choose the correct option.

Options :

6406534218095. ✖ LINE 1 generates a compilation error because of an invalid key.

The program generates the output:

6406534218096. ✓ {Arun=10200, Deepa=9300, Lekha=11200, Manoj=12200}

The program generates the output:

6406534218097. ✖ {Arun=10200, Deepa=9300, Manoj=12200, Lekha=11200}

The program generates the output:

6406534218098. ✖ {Manoj=12200, Lekha=11200, Deepa=9300, Arun=10200}

Question Number : 106 Question Id : 6406531252062 Question Type : MCQ

Correct Marks : 4

Question Label : Multiple Choice Question

The method `Stream.iterate(e, f)` returns an infinite sequential ordered `Stream` produced by iterative application of a function `f` to an initial element `e`, producing a `Stream` consisting of `e, f(e), f(f(e)), etc.`

Based on the above information, consider the code given below, and answer the question that follows.

```
import java.util.stream.Stream;

public class Test {
    public static void main(String[] args) {
        Stream.iterate(50, n -> n - 7)
            .map(n -> n / 2)
            .filter(n -> n % 4 == 0)
            .limit(3)
            .forEach((x) -> System.out.print(x + " "));
    }
}
```

What will the output be?

Options :

6406534218099. ✖ 36 8 -20 -48

6406534218100. ✔ 4 0 -20

6406534218101. ✖ 25 21 18

6406534218102. ✖ 28 32 56

Question Number : 107 Question Id : 6406531252063 Question Type : MCQ

Correct Marks : 4

Question Label : Multiple Choice Question

Consider the code given below.

```
class CricketMatch implements Cloneable {
    String teamName;
    int[] scores;

    public CricketMatch(String n, int[] s) {
        teamName = n;
        scores = s;
    }

    public Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}

public class Test {
    public static void main(String[] args) throws CloneNotSupportedException {
        int[] s = {250, 300, 275};
        CricketMatch m1 = new CricketMatch("India", s);
        CricketMatch m2 = (CricketMatch) m1.clone();
        CricketMatch m3 = m1;

        m2.scores[1] = 320;
        m3.teamName = "Australia";

        System.out.println(m1.teamName + " " + m1.scores[1]);
        System.out.println(m2.teamName + " " + m2.scores[1]);
        System.out.println(m3.teamName + " " + m3.scores[1]);
    }
}
```

What will the output be?

Options :

Australia 320

India 320

6406534218103. ✓ Australia 320

Australia 300

India 320

6406534218104. ✗ Australia 300

India 300

India 320

6406534218105. ✗ Australia 300

6406534218106. ✗

India 300
India 300
Australia 300

Question Number : 108 Question Id : 6406531252064 Question Type : MCQ

Correct Marks : 4

Question Label : Multiple Choice Question

Method `Optional.ofNullable(T value)` returns an `Optional` describing the specified value, if non-null, otherwise returns an empty `Optional`. Based on this description, consider the code given below, and answer the question that follows.

```
import java.util.*;
class HearingAid {
    String brand;
    String batteryType;

    public HearingAid(String b, String bt) {
        this.brand = b;
        this.batteryType = bt;
    }
}

public class Test {
    public static void main(String[] args) {
        var aidList = new ArrayList<HearingAid>();
        aidList.add(new HearingAid("Phonak", "Rechargeable"));
        aidList.add(new HearingAid("Widex", null));

        for (HearingAid obj : aidList) {
            Optional<String> op1 = Optional.ofNullable(obj.batteryType);
            op1.ifPresent(type -> System.out.println(type));
        }
    }
}
```

Choose the correct option.

Options :

This program generates the output:

Rechargeable

6406534218107. ✖ null

This program generates the output:

6406534218108. ✔ Rechargeable

6406534218109. ✖ This program terminates abnormally due to NullPointerException without printing any value.

6406534218110. ✖ This program terminates abnormally due to NullPointerException after printing the value:
Rechargeable

Question Number : 109 Question Id : 6406531252065 Question Type : MCQ

Correct Marks : 4

Question Label : Multiple Choice Question

Consider the code given below.

```
import java.io.*;

class AutomaticGate implements Serializable {
    private boolean isOpen;
    private transient String location;
    private transient int gCode;

    // Constructor to initialize instance variable
    public String toString() {
        return "isOpen=" + isOpen + ", location=" + location + ", gCode=" + gCode;
    }

    private void writeObject(ObjectOutputStream out) throws IOException {
        out.defaultWriteObject();
        out.writeInt(gCode + 123);
    }

    private void readObject(ObjectInputStream in) throws Exception {
        in.defaultReadObject();
        gCode = in.readInt() - 123;
    }
}

public class Test {
    public static void main(String[] args) throws Exception {
        var fos = new FileOutputStream("GateData.txt");
        var oos = new ObjectOutputStream(fos);
        AutomaticGate gate1 = new AutomaticGate(true, "North Entrance", 456789);
        oos.writeObject(gate1);

        var fis = new FileInputStream("GateData.txt");
        var ois = new ObjectInputStream(fis);
        AutomaticGate obj = (AutomaticGate) ois.readObject();
        System.out.println(obj);
    }
}
```

What will the output be?

Options :

6406534218111. ✖ isOpen=true, location=North Entrance, gCode=0

6406534218112. ✖ isOpen=true, location=North Entrance, gCode=456789

6406534218113. ✔ isOpen=true, location=null, gCode=456789

6406534218114. ✖

isOpen=true, location=null, gCode=0

Sub-Section Number : 3
Sub-Section Id : 640653194519
Question Shuffling Allowed : Yes

Question Number : 110 Question Id : 6406531252055 Question Type : MCQ

Correct Marks : 5

Question Label : Multiple Choice Question

Consider the following Java code.

```
class MastersEligibility {
    private double gpa;
    private int entranceScore;

    public MastersEligibility(double gpa, int score) {
        assert gpa >= 0 && gpa <= 10 : "GPA must be between 0 and 10";
        assert score >= 0 && score <= 100 : "Score must be between 0 and 100";

        this.gpa = gpa;
        this.entranceScore = score;
    }

    private boolean isEligible() {
        return gpa >= 7.0 && entranceScore >= 50;
    }

    public void checkEligibility() {
        assert isEligible() : "Not eligible for Master's program";
        System.out.println("Eligible for Master's program");
    }
}

class TestEligibility {
    public static void main(String[] args) {
        MastersEligibility student = new MastersEligibility(6.5, 55);
        student.checkEligibility();
    }
}
```

Choose the correct option when the class is executed as:

java -ea TestEligibility

Options :

6406534218071. ✖ This code prints: Eligible for Master's program

6406534218072. ✖ This code generates AssertionError: GPA must be between 0 and 10

6406534218073. ✖ This code generates AssertionError: Score must be between 0 and 100

6406534218074. ✔ This code generates AssertionError: Not eligible for Master's program

Question Number : 111 Question Id : 6406531252066 Question Type : MCQ

Correct Marks : 5

Question Label : Multiple Choice Question

Consider the Java code given below. Assume the file "data.txt" already contains the following text:

All the best for exam

```
import java.io.*;
public class Test {
    public static void main(String[] args) {
        try {
            FileInputStream fis = new FileInputStream("data.txt");
            int i;
            while ((i = fis.read()) != -1) {
                if (i == 'e') {
                    System.out.print('#');
                } else {
                    System.out.print((char) i);
                }
            }
            fis.close();
        } catch (IOException e) {
            System.out.println("File not found!");
        }
    }
}
```

Choose the correct option regarding the output of the program:

Options :

6406534218115. ✔ All th# b#st for #xam

6406534218116. ✖ All the best for exam

6406534218117. ✖ All th best for exam

6406534218118. ✖ File not found!

Question Number : 112 Question Id : 6406531252067 Question Type : MCQ

Correct Marks : 5

Question Label : Multiple Choice Question

Method `Collectors.partitioningBy(predicate)` returns a `Collector` which partitions the input elements according to a predicate, and organizes them into a `Map<Boolean, List<T>>`.

```
import java.util.*;
import java.util.stream.Collectors;
public class Test {
    public static void main(String[] args) {
        var numbers = new ArrayList<Integer>(); // LINE 1

        for (int i = 1; i <= 10; i++) {
            numbers.add(i);
        }

        Map<Boolean, List<Integer>> partitioned = numbers.stream()
            .collect(Collectors.partitioningBy(num -> num > 5));

        System.out.println("true -> " + partitioned.get(true));
        System.out.println("false -> " + partitioned.get(false));
    }
}
```

Choose the correct option.

Options :

This program generates the output:

true -> [1, 2, 3, 4, 5]

6406534218119. ✖ false -> [6, 7, 8, 9, 10]

This program generates the output:

true -> [6, 7, 8, 9, 10]

6406534218120. ✔ false -> [1, 2, 3, 4, 5]

This program generates compile time error at LINE 1 because the reference type of list is not specified.

6406534218121. ✖

This program generates a runtime error due to incompatible stream elements.

6406534218122. ✖

Question Number : 113 Question Id : 6406531252068 Question Type : MCQ

Correct Marks : 5

Question Label : Multiple Choice Question

Consider the Java code given below.

```
import javax.swing.*;
import java.awt.*;

public class RatingForm extends JFrame {
    JPanel pnlName, pnlRating, pnlSubmit;
    JLabel lblName, lblRating;
    JTextField txtName;
    JComboBox<String> cmbRating;
    JButton btnSubmit;

    public RatingForm() {
        lblName = new JLabel("Name:");
        lblRating = new JLabel("Rating:");
        txtName = new JTextField(15);
        cmbRating = new JComboBox<>(new String[]{"1", "2", "3", "4", "5"});
        btnSubmit = new JButton("Submit");

        pnlName = new JPanel();
        // add lblName and txtName to pnlName
        pnlRating = new JPanel();
        // add lblRating and cmbRating to pnlRating
        pnlSubmit = new JPanel();
        // add btnSubmit to pnlSubmit

        // CODE BLOCK

        setVisible(true);
        setSize(400, 300);
    }

    public static void main(String[] args) {
        new RatingForm();
    }
}
```

Choose the correct option to be filled in place of CODE BLOCK such that the above program produces the GUI given below.

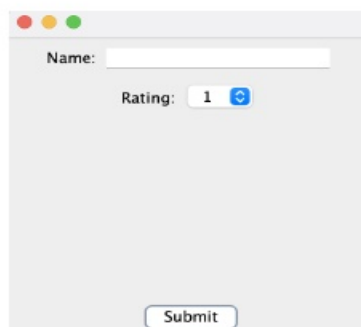


Figure 1: Rating Form

Options :

```
add(pnlName, "Center");
add(pnlRating, "North");
add(pnlSubmit, "South");
```

6406534218123. ✖

6406534218124. ✖

```
add(pnlName, "North");  
add(pnlRating, "South");  
add(pnlSubmit, "Center");
```

```
add(pnlName, "North");  
add(pnlRating, "Center");  
6406534218125. ✓ add(pnlSubmit, "South");
```

```
add(pnlName, "South");  
add(pnlRating, "Center");  
6406534218126. ✗ add(pnlSubmit, "North");
```

Sub-Section Number : 4
Sub-Section Id : 640653194520
Question Shuffling Allowed : Yes

Question Number : 114 Question Id : 6406531252069 Question Type : MSQ

Correct Marks : 4 Max. Selectable Options : 0

Question Label : Multiple Select Question

Which of the following statements is/are correct regarding garbage collection?

Options :

6406534218127. ✓ The garbage collector frees unused memory in the heap.

6406534218128. ✗ The garbage collector frees unused memory in the stack.

6406534218129. ✓ The mark-and-sweep is a common garbage collection technique.

6406534218130. ✗ The garbage collection eliminates all memory leaks in every situation.

Question Number : 115 Question Id : 6406531252070 Question Type : MSQ

Correct Marks : 4 Max. Selectable Options : 0

Question Label : Multiple Select Question

Consider the Java code given below.

```
import java.util.concurrent.ConcurrentHashMap;
class AddItemThread extends Thread {
    private ConcurrentHashMap<String, Integer> cart;
    public AddItemThread(ConcurrentHashMap<String, Integer> c) {
        cart = c;
    }
    public void run() {
        cart.put("item3", 1);
        cart.put("item4", 2);
    }
}
class ListItemThread extends Thread {
    private ConcurrentHashMap<String, Integer> cart;
    public ListItemThread(ConcurrentHashMap<String, Integer> c) {
        cart = c;
    }
    public void run() {
        for (String item : cart.keySet()) {
            System.out.println(item + ": " + cart.get(item));
        }
    }
}

public class ShoppingCart {
    public static void main(String[] args) {
        ConcurrentHashMap<String, Integer> cart = new ConcurrentHashMap<>();
        cart.put("item1", 1);
        cart.put("item2", 2);
        AddItemThread addItemThread = new AddItemThread(cart);
        ListItemThread listItemThread = new ListItemThread(cart);
        addItemThread.start();
        listItemThread.start();
        try {
            addItemThread.join();
            listItemThread.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("Final Cart: " + cart);
    }
}
```

Which of the following is true about the given code.

Options :

This program may generate the output:

item2: 2

item1: 1

item4: 2

item3: 1

6406534218131. ✓ Final Cart: {item2=2, item1=1, item4=2, item3=1}

The program may generate the output:

item2: 2

item1: 1

6406534218132. ✓

Final Cart: {item2=2, item1=1, item4=2, item3=1}

The program may generate the output:

item1: 1

item2: 2

6406534218133. ✖

Final Cart: {item1=1, item2=2}

The program always generates the output:

item3: 1

item4: 2

6406534218134. ✖

Final Cart: {item1=1, item2=2, item3=1, item4=2}

Question Number : 116 Question Id : 6406531252071 Question Type : MSQ

Correct Marks : 4 Max. Selectable Options : 0

Question Label : Multiple Select Question

Consider the code given below.

```
class ConferenceRoom {
    private boolean isRoomAvailable = true;
    public synchronized void bookRoom(String employeeName) {
        if (isRoomAvailable) {
            System.out.println(employeeName + " successfully booked the room.");
            isRoomAvailable = false;
        } else {
            System.out.println(employeeName + " could not book the room.");
        }
    }
}

class Employee implements Runnable {
    private ConferenceRoom conferenceRoom;
    private String employeeName;

    public Employee(ConferenceRoom cr, String en) {
        conferenceRoom = cr;
        employeeName = en;
    }

    public void run() {
        conferenceRoom.bookRoom(employeeName);
    }
}

public class Test{
    public static void main(String[] args) {
        ConferenceRoom room = new ConferenceRoom();
        Thread emp1 = new Thread(new Employee(room, "Hema"));
        Thread emp2 = new Thread(new Employee(room, "Geeta"));
        Thread emp3 = new Thread(new Employee(room, "Sita"));
        emp1.start();
        emp2.start();
        emp3.start();
    }
}
```

Which of the following options is/are possible result/s of the above code?

Options :

This code generates the output:

Hema successfully booked the room.

Geeta successfully booked the room.

Sita could not book the room.

6406534218135. ✖

6406534218136. ✔

This code generates the output:

Hema successfully booked the room.

Sita could not book the room.

Geeta could not book the room.

This code generates the output:

Sita successfully booked the room.

Hema could not book the room.

6406534218137. ✓ Geeta could not book the room.

This code generates the output:

Hema successfully booked the room.

Sita successfully booked the room.

6406534218138. ✗ Geeta successfully booked the room.

Question Number : 117 Question Id : 6406531252072 Question Type : MSQ

Correct Marks : 4 Max. Selectable Options : 0

Question Label : Multiple Select Question

Consider the Java code given below that processes `Displayable` objects in a shop. From among the options, identify the appropriate function header for function `displayAll` that takes as input a collection of `Displayable` objects and calls the `display` method on each.

```
import java.util.*;
interface Displayable {
    void display();
}
class Electronics implements Displayable {
    // Constructor
    // method display() that prints electronic item details
}
class Clothing implements Displayable {
    // Constructor
    // method display() that prints clothing item details
}
public class Shop {
    // LINE 1: FUNCTION HEADER
    {
        // invokes display() on each element
    }

    public static void main(String[] args) {
        List<Displayable> items = new ArrayList<>();
        items.add(new Electronics("Laptop", 50000));
        items.add(new Clothing("T-Shirt", "Medium"));
        displayAll(items);
    }
}
```

Choose the correct option(s).

Options :

6406534218139. ✖ `public static void displayAll(Collection<?> elements)`

6406534218140. ✖ `public static <T> void displayAll(Collection<T> elements)`

6406534218141. ✔ `public static <T extends Displayable> void displayAll(Collection<T> elements)`

6406534218142. ✔ `public static void displayAll(List<Displayable> elements)`

Sub-Section Number :

5

Sub-Section Id :

640653194521

Question Shuffling Allowed :

Yes

Question Number : 118 Question Id : 6406531252073 Question Type : MSQ

Correct Marks : 6 Max. Selectable Options : 0

Question Label : Multiple Select Question

Consider the following Java code.

```
1. class Animal {  
2.     protected final void makeSound() {  
3.         System.out.println("Animal makes sound");  
4.     }  
5. }  
6. class Bird {  
7.     public void fly() {  
8.         System.out.println("Bird flies");  
9.     }  
10. }  
11. class Dog extends Animal{  
12.     public void makeSound(){  
13.         System.out.println("Dog barks");  
14.     }  
15. }  
16. class Parrot extends Animal, Dog{  
17. }  
18. public class Test {  
19.     public static void main(String[] args) {  
20.         Parrot p1 = new Animal();  
21.         Animal a1 = new Dog();  
22.     }  
23. }
```

Identify the line/s which has/have errors.

Options :

6406534218143. ✖ Line 2

6406534218144. ✖ Line 7

6406534218145. ✖ Line 11

6406534218146. ✔ Line 12

6406534218147. ✔ Line 16

6406534218148. ✔ Line 20

6406534218149. ✖ Line 21

Question Number : 119 Question Id : 6406531252074 Question Type : MSQ

Correct Marks : 6 Max. Selectable Options : 0

Question Label : Multiple Select Question

Consider the Java code given below.

```
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;

class TokenSystem extends JFrame implements ActionListener {
    JButton btnTakeToken, btnCancelToken;
    JLabel label;
    JPanel p1, p2;

    public TokenSystem() {
        p1 = new JPanel();
        p2 = new JPanel();

        btnTakeToken = new JButton("Take Token");
        btnCancelToken = new JButton("Cancel Token");
        label = new JLabel("Please select an option");

        p1.add(btnTakeToken);
        p1.add(btnCancelToken);
        p2.add(label);

        add(p1, "Center");
        add(p2, "South");

        btnTakeToken.setActionCommand("TakeToken");
        btnCancelToken.setActionCommand("CancelToken");
        btnTakeToken.addActionListener(this);
        btnCancelToken.addActionListener(this);

        setSize(350, 150);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        // CODE SEGMENT
    }
}

public class HospitalQueue {
    public static void main(String[] args) {
        new TokenSystem();
    }
}
```

Choose the correct code segment to be filled inside the method `actionPerformed()` such that on clicking the Take Token button, the label text changes to Token issued. Please wait., and on clicking the Cancel Token button, the label text changes to Token canceled..



Options :

6406534218150. ✓

```

if (e.getActionCommand().equals("TakeToken"))
    label.setText("Token issued. Please wait.");
else if (e.getActionCommand().equals("CancelToken"))
    label.setText("Token canceled.");

```

6406534218151. ✗

```

if (e.getSource().equals("TakeToken"))
    label.setText("Token issued. Please wait.");
else if (e.getSource().equals("CancelToken"))
    label.setText("Token canceled.");

```

6406534218152. ✓

```

if (e.getSource().equals(btnTakeToken))
    label.setText("Token issued. Please wait.");
else if (e.getSource().equals(btnCancelToken))
    label.setText("Token canceled.");

```

6406534218153. ✗

```

if (e.getActionCommand().equals(TakeToken))
    label.setText("Token issued. Please wait.");
else if (e.getActionCommand().equals(CancelToken))
    label.setText("Token canceled.");

```

Appdev2

Section Id :	64065387636
Section Number :	6
Section type :	Online
Mandatory or Optional :	Mandatory
Number of Questions :	31
Number of Questions to be attempted :	31
Section Marks :	100